

TEST PLAN REPORT

SecSuite — Unified Cybersecurity Suite

Version 1.0 | 2025

Prepared by:

Umut Bayram · Archyn Mikhailov · Kıvanç Taş · Yiğit Sarı

Project URL: <https://githappensstobb-coder.github.io/>

1. Introduction

This Test Plan Report describes the testing strategy, scope, methodology, environment, and test cases for SecSuite — a modular, web-based cybersecurity suite. The plan ensures that every functional requirement and non-functional quality attribute defined in the Project Analysis Report and Project Specifications Report is systematically verified before the system is released.

The primary objective of testing is to confirm that SecSuite correctly identifies online threats across its four core detection modules, performs within the defined service-level agreement (SLA) thresholds, enforces security controls, and provides a reliable, user-friendly experience across supported platforms.

2. Project Scope

2.1 Project Title

SecSuite — Unified Cybersecurity Suite

2.2 Brief Description of the System

SecSuite is a serverless, edge-deployed web platform that consolidates four independent cybersecurity tools into a single unified interface. It leverages state-of-the-art machine learning and deep learning models to assess digital threats and returns explainable results to users ranging from casual internet users to security analysts and journalists.

The system is built on a TypeScript-first stack: a SvelteKit/Tailwind CSS frontend, a Hono.js API running on Cloudflare Workers, Cloudflare D1 (SQLite) for persistent storage, Cloudflare KV for hot caching, and a self-hosted AI inference engine running ONNX models. Analysis requests are processed asynchronously via a message queue with Server-Sent Events (SSE) delivering real-time progress to clients.

2.3 Main Functionalities of the System

The following four core analysis modules are in scope for testing:

- **Social Media Bot Detection** — Accepts a username or profile URL; analyses network graph and behavioural patterns using Graph Neural Networks (GNNs); returns a Bot/Human classification with a confidence score and D3.js force-directed graph visualisation.
- **Phishing Email Detection** — Accepts email headers and body text; identifies high-risk elements such as suspicious embedded links and urgent language; returns a safety score and list of flagged indicators.
- **Fake News Verification** — Accepts raw article text or a news URL; extracts text from the URL and performs credibility assessment using NLP-based linguistic analysis; returns a credibility score and source evaluation.

- Malicious Link Evaluation — Accepts a URL; performs DNS/WHOIS lookups and queries threat intelligence databases (including Google Safe Browsing); renders a safe screenshot of the destination; returns a Safe/Malicious verdict with a full redirect chain and SSL status.

Cross-cutting capabilities that are also in scope include user authentication (email/password, social OAuth, magic link, anonymous guest sessions), analysis history and visibility controls, API key management, rate limiting and quota enforcement, caching and deduplication, SSE real-time streaming, and CAPTCHA bot-detection protection.

3. Features to Be Tested

3.1 Bot Detection Module

- Correct Bot/Human classification for known bot and human accounts.
- Confidence score and network graph visualisation rendered.
- API response time ≤ 10 seconds per requirement.
- KV cache hit returns results without invoking the AI engine.
- Graceful handling of non-existent or private usernames.

3.2 Phishing Email Detection Module

- Accurate identification of phishing vs. legitimate email content.
- High-risk elements (suspicious links, urgent language) highlighted.
- Safety score returned with classification.
- API response time ≤ 8 seconds.
- Validation error on empty email body.
- Embedded malicious links detected and flagged independently.

3.3 Fake News Verification Module

- Credibility assessment of known credible and fabricated news content.
- URL-based article extraction and text analysis.
- Credibility score and bias/source indicators returned.
- API response time ≤ 8 seconds.
- Graceful failure for inaccessible or 404 URLs.
- Validation error on empty text/URL input.

3.4 Malicious Link Evaluation Module

- Safe/Malicious verdict for known malicious URLs (Google Safe Browsing DB).
- Safe screenshot of destination rendered.
- Full redirect chain displayed.
- WHOIS domain age, SSL certificate status shown.
- API response time ≤ 8 seconds.

- Validation error on malformed URL input.

3.5 Authentication and User Management

- User registration with email/password; duplicate email rejection.
- Login with valid and invalid credentials; JWT issuance.
- Social OAuth login (Google, and other providers).
- Magic link passwordless authentication.
- Anonymous guest session creation and analysis accumulation.
- Anonymous-to-registered account transition with full history transfer.
- API key creation, listing, and revocation.

3.6 Rate Limiting and Security

- HTTP 429 returned with Retry-After header when quota is exceeded (anonymous: 100/day, authenticated: 500/day).
- CAPTCHA validation on registration, sign-in, password reset, and guest analysis endpoints.
- Input sanitisation — SQL injection and XSS attempts blocked.
- TLS 1.2+ enforcement on all API traffic.

3.7 Caching and Deduplication

- KV hot cache (1-hour TTL) returns results for identical inputs within the window.
- DB cache fallback for analyses within 24 hours; KV back-populated on hit.
- SHA-256 hash deduplication of normalised inputs across all cache layers.

3.8 Performance and Availability

- System sustains ≥ 100 concurrent users without 5xx errors.
- System uptime $\geq 99.5\%$ over a 24-hour monitoring period.

3.9 Analysis History and Visibility

- History persistence for authenticated users across sessions.
- Private visibility — analysis invisible to other users and search.
- Public visibility — analysis findable in public search.
- SSE stream delivers phase-by-phase progress events to subscribing clients.

4. Testing Methodology

The following testing levels and types are applied throughout the SecSuite project lifecycle. All automated tests are implemented using Vitest targeting a minimum 90% code coverage, as mandated by the project's pseudo-requirements.

4.1 Unit Testing

Individual functions and modules are tested in isolation with mocked dependencies. Unit tests cover input normalisation and SHA-256 hashing routines, threat-score aggregation logic in the Scoring Engine, rate-limiter sliding-window calculations, authentication token generation and validation utilities, and DrizzleORM data-access layer functions.

4.2 Integration Testing

Integration tests verify the correct interaction between connected subsystems. Key integration scenarios include the end-to-end flow from API gateway through the Queue to the AI Engine and back; the two-tier caching strategy (KV → relational DB) including cache back-population; the anonymous-to-authenticated account transition with history migration; and webhook delivery including HMAC signing and retry logic.

4.3 System Testing

The complete, deployed system is tested against all functional and non-functional requirements. System tests are executed against the staging environment (Cloudflare Workers preview deployment) and cover all four detection modules, authentication flows across all supported methods, API key management, and visibility control behaviour.

4.4 Performance Testing

Performance tests evaluate speed, throughput, and resource utilisation against the defined SLA targets. Load tests simulate 100 concurrent users and 500 concurrent requests. Response-time assertions are applied per module: bot detection ≤ 10 s, all other modules ≤ 8 s. Uptime is monitored continuously over a minimum 24-hour window targeting $\geq 99.5\%$ availability.

4.5 Security Testing

Security tests verify that all protection mechanisms operate correctly. Tests include CAPTCHA validation on protected endpoints, SQL injection and XSS input-sanitisation checks, TLS enforcement, rate-limit exhaustion behaviour, and API key hash verification. Inputs are tested for injection payloads to confirm that the system sanitises and rejects malicious content without exposing internal errors.

4.6 User Acceptance Testing (UAT)

UAT scenarios are executed with representative users drawn from the three primary personas defined in the Analysis Report. An end user tests the phishing and link-checking tools using a realistic suspicious-email scenario. A journalist tests the bot detection tool on a social media account of interest. A social media analyst reviews the authenticated history dashboard and verifies trend visualisation. Feedback is collected and any blocking usability issues are addressed before final release.

5. Test Environment

5.1 Hardware

- Developer workstations: x86-64 machines running Windows 11, macOS Ventura, or Ubuntu 22.04 LTS.
- Cloud inference server: GPU-enabled virtual machine (minimum NVIDIA T4) for AI Engine model loading and inference performance tests.
- Edge network: Cloudflare Workers global edge (staging environment) — no dedicated on-premises server required.
- Mobile devices (UAT): iOS 16+ and Android 12+ smartphones for responsive-design validation.

5.2 Software

Operating Systems: Windows 11, macOS 13 (Ventura), Ubuntu 22.04 LTS.

Runtime and Language: Node.js 20 LTS; TypeScript 5.x for both frontend (SvelteKit) and backend (Hono.js).

Frontend Framework: SvelteKit with Tailwind CSS; D3.js for force-directed graph visualisation.

Backend/API: Hono.js on Cloudflare Workers; DrizzleORM with Cloudflare D1 (SQLite); Cloudflare KV for caching; Cloudflare Queues for async processing.

AI/ML: Python 3.11; PyTorch / DGL for GNN models; scikit-learn for ensemble classifiers; ONNX Runtime for edge inference; Cloudflare Workers AI for model serving.

Test Framework: Vitest (unit and integration); Playwright (end-to-end and browser extension); k6 (performance and load testing).

Other Tools: Git (version control); Docker (containerisation of AI Engine); Postman / HTTP client for API exploratory testing; Wrangler CLI for Cloudflare Workers deployment and testing.

6. Test Cases

Each test case is assigned a unique Test ID. The Status column reflects expected outcomes under a correctly functioning system. Tests are grouped by functional area for clarity.

Test ID	Description	Input(s)	Expected Output	Status	Notes
TC-01	Valid username — human account	Username: @authentic_user (real, active account)	Bot Likelihood < 30%; classification: Human; network graph rendered	Pass	Bot Detection

Test ID	Description	Input(s)	Expected Output	Status	Notes
TC-02	Known bot account detection	Username: @known_bot_account (test bot seed)	Bot Likelihood $\geq 70\%$; c	Pass	Bot Detection
TC-03	Non-existent username	Username: @xyznonexistent123abc	Error message: 'User not found'; no score displayed	Pass	Bot Detection
TC-04	Response time within SLA	Any valid username submitted	Response returned within 10 seconds (SLA requirement)	Pass	Bot Detection
TC-05	Bot Detection — KV cache hit	Same username submitted twice within 24 h	Second request returns immediately from cache; no new API call	Pass	Bot Detection
TC-06	Known phishing email body	Email body containing urgent wire transfer request with spoofed domain	Safety Score ≥ 70 ; high-risk elements highlighted; classification: Phishing	Pass	Phishing
TC-07	Legitimate email body	Standard corporate newsletter from a known domain	Safety Score < 30 ; classification: Safe; no high-risk indicators	Pass	Phishing
TC-08	Empty email body submitted	Headers provided; body left empty	Validation error: 'Email body is required'	Pass	Phishing
TC-09	Email with embedded malicious link	Body containing a link matching a threat-DB entry	Link flagged as malicious; score elevated; link highlighted in results	Pass	Phishing
TC-10	Phishing detection response time	Any well-formed phishing email input	Response returned within 8 seconds (SLA requirement)	Pass	Phishing
TC-11	Verified credible article URL	URL of a Reuters or BBC news article	Credibility Score ≥ 70 ; classification: Credible; low bias indicators	Pass	Fake News
TC-12	Known fake news article text	Raw text from a flagged misinformation source	Credibility Score < 40 ; classification: Unreliable; linguistic anomalies flagged	Pass	Fake News
TC-13	Inaccessible URL submission	URL returning HTTP 404	Error message: 'Unable to retrieve article'; graceful failure shown	Pass	Fake News
TC-14	Raw text input without URL	Pasted paragraph of fabricated news text	Text extracted and assessed; credibility score returned within 8 seconds	Pass	Fake News
TC-15	Empty text field	No input submitted	Validation error: 'Please enter text or a URL'	Pass	Fake News
TC-16	Known malicious URL	URL flagged in Google Safe Browsing DB	Status: Malicious (red banner); threat category displayed; screenshot blurred	Pass	Link Checker
TC-17	Safe URL check	https://	Status: Safe (green	Pass	Link

Test ID	Description	Input(s)	Expected Output	Status	Notes
		www.google.com	banner); WHOIS age shown; SSL valid; screenshot rendered		Checker
TC-18	URL with redirect chain	Shortened URL redirecting multiple times	Full redirect chain displayed; final destination evaluated	Pass	Link Checker
TC-19	Malformed URL input	Input: 'not-a-url'	Validation error: 'Invalid URL format'	Pass	Link Checker
TC-20	Link Checker response time	Any valid URL	Response returned within 8 seconds (SLA requirement)	Pass	Link Checker
TC-21	User registration — valid data	New email + strong password	Account created; verification email sent; redirect to dashboard	Pass	Auth
TC-22	User registration — duplicate email	Email already registered	Error: 'Email already in use'; no duplicate account created	Pass	Auth
TC-23	Login — valid credentials	Registered email + correct password	Session created; JWT issued; user redirected to dashboard	Pass	Auth
TC-24	Login — invalid credentials	Registered email + wrong password	Error: 'Invalid credentials'; no session created	Pass	Auth
TC-25	Social OAuth login (Google)	OAuth flow initiated; valid Google account	User authenticated; session created; profile populated	Pass	Auth
TC-26	Anonymous guest session creation	Guest clicks 'Continue as guest'	Temporary session created; analysis history begins accumulating	Pass	Auth
TC-27	Anonymous → Registered transition	Guest with history registers a permanent account	All analysis history transferred; anonymous account deleted; zero data loss	Pass	Auth
TC-28	API key creation	Authenticated user creates a new API key	Key returned once (plaintext); hash stored; key visible in dashboard	Pass	Auth
TC-29	Rate limit — anonymous user exceeded	Anonymous user sends > 100 requests/day	HTTP 429 returned; Retry-After header included	Pass	Security
TC-30	Rate limit — authenticated user	Authenticated user sends > 500 requests/day	HTTP 429 returned after quota exhausted	Pass	Security
TC-31	CAPTCHA validation on registration	Registration attempt from automated script without CAPTCHA token	Request rejected; HTTP 400 returned	Pass	Security
TC-32	SQL injection in input field	Input: ' OR 1=1; --	Input sanitized; no database error; safe error message returned	Pass	Security

Test ID	Description	Input(s)	Expected Output	Status	Notes
TC-33	XSS attempt in text fields	Input: <code><script>alert('xss')</script></code>	Input escaped; no script execution; content displayed safely	Pass	Security
TC-34	TLS enforcement	HTTP request to API endpoint	Automatic redirect to HTTPS; all data in transit encrypted	Pass	Security
TC-35	KV hot cache deduplication	Same URL submitted twice within 1 hour	Second response served from KV cache; latency < 200 ms	Pass	Caching
TC-36	DB cache fallback on KV miss	Analysis older than 1 h but within 24 h resubmitted	Result retrieved from DB; KV back-populated; correct result returned	Pass	Caching
TC-37	Concurrent user load test	100 simultaneous users submitting link-check requests	All requests completed successfully; no 5xx errors; SLA maintained	Pass	Performance
TC-38	System uptime monitoring	Continuous uptime check over 24-hour period	Uptime ≥ 99.5% (SLA requirement)	Pass	Performance
TC-39	History persistence for auth users	Authenticated user submits 5 analyses then navigates to History	All 5 analyses appear in history with correct metadata	Pass	History
TC-40	Visibility — private analysis	Analysis set to Private by owner	Only owner can view; not returned in public search	Pass	History
TC-41	Visibility — public analysis	Analysis set to Public	Any user can view via direct link and public search	Pass	History
TC-42	SSE real-time progress stream	User submits analysis and monitors progress stream	Progress events emitted for each phase: queued, processing, completed	Pass	Streaming

7. Requirements Traceability

The table below maps every functional and non-functional requirement of SecSuite to the test cases that verify it. All 42 test cases are covered.

Req. ID	Requirement Description	Test Case(s)
FR-01	System shall classify a social media account as Bot or Human using GNN-based behavioural and network analysis, returning a confidence score and force-directed graph visualisation	TC-01, TC-02, TC-03, TC-04, TC-05
FR-02	System shall detect phishing email content, highlight high-risk elements (suspicious links, urgent language), and return a safety score with classification	TC-06, TC-07, TC-08, TC-09, TC-10
FR-03	System shall perform credibility assessment of news content (raw text or URL), returning a credibility score, source evaluation, and linguistic	TC-11, TC-12, TC-13, TC-14, TC-15

Req. ID	Requirement Description	Test Case(s)
	anomaly indicators	
FR-04	System shall evaluate a URL for malicious content using DNS/WHOIS lookups and threat intelligence databases, displaying a Safe/Malicious verdict, redirect chain, SSL status, and safe screenshot	TC-16, TC-17, TC-18, TC-19, TC-20
FR-05	System shall support user registration and login via email/password, social OAuth, magic link, and anonymous guest sessions, including anonymous-to-registered account transition and API key management	TC-21, TC-22, TC-23, TC-24, TC-25, TC-26, TC-27, TC-28
FR-06	System shall enforce rate limits (anonymous: 100/day, authenticated: 500/day), return HTTP 429 with Retry-After on quota exhaustion, validate CAPTCHA on protected endpoints, sanitise inputs against SQL injection and XSS, and enforce TLS 1.2+ on all traffic	TC-29, TC-30, TC-31, TC-32, TC-33, TC-34
FR-07	System shall implement a two-tier caching strategy: KV hot cache (1-hour TTL) for identical inputs and DB cache fallback (24-hour window) with SHA-256 hash deduplication and KV back-population on DB hit	TC-05, TC-35, TC-36
NFR-01	Performance: Bot Detection API response ≤ 10 s; all other module APIs ≤ 8 s; system sustains ≥ 100 concurrent users without 5xx errors; uptime $\geq 99.5\%$ over any 24-hour window	TC-04, TC-10, TC-14, TC-20, TC-37, TC-38
NFR-02	Analysis history shall persist for authenticated users across sessions; private analyses shall be invisible to other users and excluded from public search; public analyses shall be findable via direct link and public search; SSE shall deliver phase-by-phase progress to subscribing clients	TC-39, TC-40, TC-41, TC-42

8. Risks and Limitations

8.1 Technical Risks

- **Third-Party API Rate Limits:** The Social Media Bot Detection module depends on the X (Twitter) and Reddit APIs. Rate limits or access restrictions imposed by these platforms may prevent execution of full integration tests or produce incomplete graph data during testing.
- **AI Engine Availability:** The self-hosted AI inference service requires GPU compute. Unavailability of suitable GPU infrastructure during the testing phase may force fallback to CPU inference, significantly increasing response times and potentially causing SLA tests to fail for reasons unrelated to the application logic.
- **Model Non-Determinism:** Machine learning models may produce slightly different confidence scores across runs due to floating-point variability or model updates. Test assertions for score-based pass/fail thresholds must accommodate a tolerance margin ($\pm 5\%$).

- CAPTCHA in Automated Testing: Invisible CAPTCHA challenges cannot be solved by automated test scripts. End-to-end tests that exercise CAPTCHA-protected endpoints (registration, guest analysis) require a test-mode bypass token configured in the staging environment.
- SSE Stream Timing: Real-time SSE progress event tests are inherently time-sensitive. Flaky test results may occur if the AI Engine is under load during testing, causing events to arrive outside the expected time windows.
- Cloudflare Platform Constraints: Some Cloudflare Workers platform limitations (e.g., CPU time limits of 50 ms per request for lightweight operations, KV eventual consistency) may affect the behaviour of caching and rate-limiting tests in edge cases.

8.2 Dataset and Hardware Limitations

- Training Dataset Access: The TwiBot-22, phishing email, fake news, and malicious URL datasets used for model training are large (105 GB). Full dataset loading for regression testing requires sufficient cloud storage and may incur costs beyond the project budget.
- Seed Data for Bot Tests: Creating realistic bot and human test accounts on live social media platforms is subject to platform terms of service. Tests must rely on pre-approved seed accounts or anonymised dataset snapshots rather than live scraping.
- GPU Resource Availability: Shared GPU instances on cloud providers may introduce latency variability. Performance SLA tests should be run during off-peak hours to obtain representative measurements.
- Browser Extension Testing: Automated testing of the browser extension requires a real Chromium or Firefox browser instance. Headless execution of extension tests has known limitations and may not fully replicate user interactions in the extension popup.
- Mobile Device Coverage: UAT is limited to the devices available within the team. Testing on a wider range of mobile screen sizes and operating system versions may uncover responsive-design issues not caught during development.

9. Conclusion

This Test Plan provides a comprehensive, structured framework for verifying every aspect of the SecSuite cybersecurity platform. The plan covers 42 explicit test cases spanning all four detection modules (Bot Detection, Phishing Email Analysis, Fake News Verification, and Malicious Link Evaluation), as well as authentication, security controls, caching, performance, and analysis history features.

By combining unit, integration, system, performance, security, and user acceptance testing — all backed by Vitest with a 90% code-coverage target — the plan ensures that SecSuite meets its functional requirements, adheres to its non-functional SLA thresholds (response times, uptime, concurrency), and enforces the security controls mandated by GDPR and KVKK compliance obligations.

Identified risks, including third-party API rate limits, AI model non-determinism, CAPTCHA automation constraints, and GPU resource variability, are acknowledged and mitigated through test-environment configuration (staging bypass tokens, tolerance margins, off-peak scheduling).

These mitigations allow the test suite to deliver meaningful coverage without compromising the validity of results.

Successful execution of this plan will provide the development team and project stakeholders with high confidence that SecSuite is ready for deployment and capable of delivering accurate, explainable, low-latency threat assessments to its target users.